Cerebro GPU Enablement Initiative

H100-Accelerated Training Program Proposal

Prepared for: Writer Security Engineering, SIRT, and Platform Leadership

Prepared by: Jonathan Haas (jonathan.haas@writer.com)
Date: October 23, 2025

WRITER

Strengthening Writer security operations with GPU-native workflows

Executive Summary

Writer's internal Cerebro stack already automates daily summaries, telemetry health, and attack-path analytics across `src/cerebro/automation`, `docs/runbooks`, and the SIRT playbooks. Redirecting idle H100 capacity into these pipelines lets us harden detections, accelerate investigations, and keep graph-model drift in check without relaxing guardrails. This six-week enablement track retrains responders on GPU-backed workflows, extends our automation coverage, and bakes GPU exercises into on-call readiness so security outcomes improve measurably across daily operations.

1 Why Invest Now

- **Live corpus fine-tunes:** H100-backed QLoRA adapters trained on Writer's incident transcripts and audit trails (the same payloads surfaced by `generate_daily_summary` in `src/cerebro/automati` sharpen remediation drafts used in review queues, eliminating the rewrite churn flagged during recent tabletop exercises.
- **Instant context retrieval:** Offloading embedding batches from `AgentMemoryStore` to GPUs trims long-tail recall latency from multi-minute CPU jobs to sub-second fan-out, meaning the SIRT analyst actually sees prior session memory in time to shape containment decisions.
- **Dynamic blast-radius scoring:** GPU-accelerated recomputation of the attack-path models in `src/cerebro/attack_path` keeps the blast-radius tool aligned with nightly drift, so the detections shipped through `docs/runbooks/benchmark-response.md` stay relevant to the current IAM graph.
- **Operational resilience:** Routine enablement scenarios that reserve GPU capacity exercise Pulumi deployment hooks, confirm monitoring signals in the telemetry dashboards runbook, and give security on-call a rehearsed GPU break-glass plan instead of experimenting during incidents.

2 Program Objectives

- Equip Writer security engineers, threat hunters, and SIRT with GPU-enhanced Cerebro runbooks that plug into existing automation endpoints (daily summaries, telemetry health, attack-path simulators).
- Operationalize automated H100 training and inference pipelines inside the current Pulumi stacks and GitHub workflows so QLoRA, embedding, and graph jobs are monitored alongside CPU services.
- Establish success metrics for remediation velocity, investigation depth, and blast-radius accuracy, instrumented via the automation tests in `tests/automation` and surfaced in telemetry dashboards.
- Produce reusable internal collateral (lab guides, regression harnesses, dashboards) that map directly to Writer security guild rituals and on-call rotations.

3 Operational Benefit Deep Dives

GPU-Tuned Remediation Drafts

Daily summary automations (`src/cerebro/automation/daily_summary.py`) already seed agent sessions with hot findings; finetuning on those transcripts with H100s improves the downstream suggestions generated by `review_service.py`, closing the loop that agents use before on-call approvals. The program instruments before/after rewrite counts so security directors see concrete uplift.

Accelerated Memory Recall

`AgentMemoryStore` currently leans on CPU hashing and optional OpenAI embeddings. A GPU FAISS pipeline slotted behind the same interface drops lookup latency and enables larger context windows without rewriting the caller APIs that live in `memory_session.py` and the benchmarking harnesses. Labs walk engineers through swapping the backend and verifying latency via the automation tests.

Continuous Graph Risk Refresh

The attack-path models in `src/cerebro/attack_path` feed detections and the blast-radius tooling used in SIRT drills. GPU-accelerated recomputation aligned with `benchmark-response.md` keeps drift under control, and the curriculum demonstrates how to validate fresh graphs inside the existing regression notebooks and CI workflows.

Telemetry Integrity

Scripts such as `scripts/analyze_frontend_events.py` and the telemetry runbook depend on timely aggregations; GPU-backed batch jobs let us widen observation windows without missing the 24h freshness checks. Participants observe the impact directly in the dashboards referenced in `telemetry-dashboards.md`.

4 Audience

(Primary Cohorts)

Cohort	Focus	Expected Outcomes
Security Engineering	Build and automate finetune + embedding jobs	Own scheduled GPU workloads and evaluate outputs
Security Incident Response (SIRT)	Apply GPU-accelerated findings in tabletop and live drills	Reduce time-to-mitigation via enhanced agent outputs
Threat Research / Detection Engineering	Tune attack-path analytics and validate detection content	Deliver fresher graph-derived detections and tuning feedback

5 Curriculum Overview

Phase 1: Foundations (Week 1)

- GPU architecture briefing and Cerebro AI roadmap.
- Hands-on: Configuring Terraform/Pulumi hooks for dedicated H100 queues.
- Success criterion: participants deploy a sandbox pipeline referencing production telemetry exports.

Phase 2: Finetune Ops (Weeks 2–3)

- Lab: QLoRA adapters on incident transcripts exported from automation jobs, promoted through the staging review queue that exercises `review_service.py`.
- Workshop: Bias/variance controls, safety guardrails, and automated regression tests wired into existing GitHub Actions.
- Success criterion: 10%+ improvement in draft acceptance measured via `tests/automation` harness and review queue audit logs.

Phase 3: Memory Acceleration (Week 4)

- Lab: GPU FAISS pipeline for embeddings integrated with `src/cerebro/agents/memory` and validated against `AgentMemoryStore` integration tests.
- Benchmark: Compare recall latency, retrieval depth, and token cost across org footprints captured in our telemetry datasets.
- Success criterion: Sub-5s recall for 95th percentile memory lookups while keeping storage budgets flat.

Phase 4: Graph Intelligence (Week 5)

- Lab: Port attack-path analytics (`src/cerebro/query` and `attack_path`) to cuGraph/PyG.
- Simulation: Inject drift scenarios aligned with benchmark regression fixtures and monitor risk re-ranking.
- Success criterion: 3x faster recomputation and improved top-10 precision with updated detections piped into automation alerts.

Phase 5: Integration Showcase (Week 6)

- Cross-team tabletop: End-to-end incident run leveraging GPU-enhanced features.
- Artifact sprints: Produce updated runbooks, SOC dashboards, and internal walkthrough recordings.
- Success criterion: Executable incident rehearsal and collateral package for Writer security on-call refresh.

6 Resource Plan

- **Compute:** 4 dedicated H100s for nightly finetunes, 2 for daytime labs; orchestrated through existing Pulumi stacks with GPU-capacity tags and monitored via the same Prometheus alerts we use for API workers.
- **Data:** Sanitized agent transcripts, synthetic attack-path datasets, telemetry summaries from `scripts/daily_agent_summary.py`, and curated warehouse exports referenced in the telemetry runbook.
- **Staffing:** Enablement lead (program owner), MLOps engineer, SIRT facilitator, detection engineering partner, plus a platform SRE liaison for Pulumi updates.
- **Budget:** Primarily internal opportunity cost; optional offsite workshop expenses for the security guild summit and incremental storage for GPU checkpoints.

7 Risk and Control Considerations

- **Data handling:** Training sets exclude regulated customer data and follow the sanitization pipeline already enforced for automation exports; compliance verification is built into Phase 1 labs.
- **Access controls:** GPU queues inherit IAM scopes from the Pulumi stacks, and labs include a validation step that cross-checks CEL policies for destructive tools before enabling GPU-backed runs.
- **Fallback readiness:** Each GPU workflow documents a CPU fallback path so the automation suite maintains baseline coverage if H100 capacity is reclaimed for production incidents.
- **Observability parity:** Metrics emitted by new GPU jobs mirror the existing automation counters, enabling security leadership to compare historical trends without schema changes.

8 Evaluation Metrics

- **Remediation velocity:** Median review queue resolution time pre/post training.
- **Investigation depth:** Number of unique memory artifacts cited per incident.
- **Blast-radius accuracy:** Precision/recall of top-risk paths in validation scenarios.
- **Adoption:** Percentage of participants scheduling recurring GPU workloads within 30 days.

9 Next Steps

1. Approve program charter and allocate H100 capacity (Week 0).
2. Stand up shared lab environment and data snapshots (Week 0).
3. Launch Phase 1 sessions and begin weekly reporting cadence.
4. Review outcomes at midpoint (end of Week 3) with metrics pulled from automation dashboards and adjust curriculum if needed.
5. Deliver final showcase assets and success report to Writer security leadership (end of Week 6), including benchmark diffs and on-call readiness checklist updates.

Appendix: Tooling Stack

- **Training:** Hugging Face `transformers`, `peft`, `bitsandbytes`, with UV-managed environments and Writer's secrets pipeline.
- **Vector:** FAISS-gpu, RAPIDS cuDF/cuML for preprocessing, orchestrated through Celery workers that already handle memory ingests.
- **Graphs:** RAPIDS cuGraph and PyTorch Geometric with CUDA acceleration, integrated with our attack-path regression notebooks.
- **Observability:** Extended Prometheus dashboards for GPU utilization, model performance drift, and Pulumi health checks.